



Dokumentace projektu z kursu IVH
Grafický čítač na FPGA a ZYNQu

Brno, 14. května 2024

David Kvaček
xkvace00@stud.fit.vutbr.cz

1 Téma zadání

Předmětem projektu je implementace zobrazování na maticovém displeji s využitím FPGA. Ovládání je řešeno pomocí vnitřního konečného stavového automatu (FSM). Realizace vlastního zobrazování je uskutečněna prostřednictvím maticového displeje o rozměrech 8×8 buněk.

Veškerá funkčnost aplikace není ovlivněna žádným uživatelským vstupem, což značně zjednodušuje implementaci. Uživatel není nucen k jakékoli interakci a jedná se tedy o pouhé spuštění, případně nahrání programu do FPGA kitu.

2 Chování aplikace

Opakovaně se na maticovém displeji bude zobrazovat striktně definovaná smyčka:

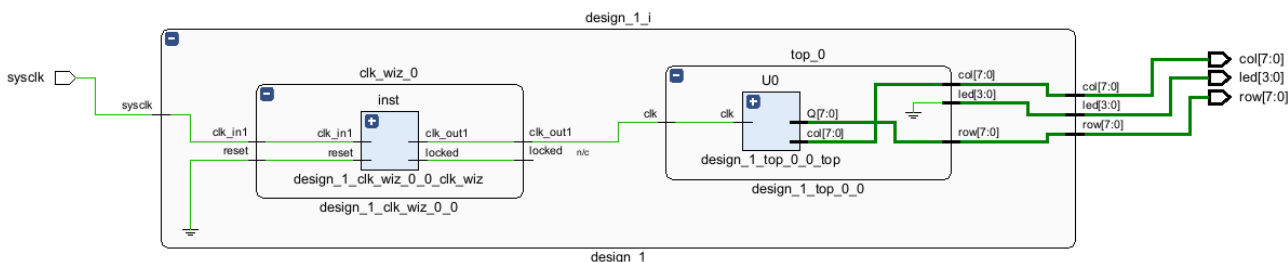
1. dvouciferné dekadické číslo od 00 do 20, které se každou sekundu inkrementuje (délka kroku 1 s),
2. dvouciferné dekadické číslo od 21 do 30, které je zobrazeno inverzně,
3. vlastní obrázek, který se zobrazí po dobu 5 sekund,
4. libovolný způsob animace, která je postupně promítnuta na displeji.

3 Blokový návrh

Návrh blokového schématu obsahuje dvě hlavní entity. Nejdůležitější částí celého projektu je bezpochyby entita nejvyšší úrovně (`top.vhd`). Pro jednoduchost je veškerá funkcionality realizována právě v tomto souboru.

Díky tomu je blokový návrh jednoduchý a vedle entity nejvyšší úrovně obsahuje pouze tzv. *Clocking wizard*, který slouží k nastavení generování hodinového signálu o předem dané frekvenci (v tomto případě 25 MHz). Vstupem pro tuto jednotku je systémový hodinový signál (`sysclk`), jenž je transformován na požadovanou frekvenci.

Výstupem Clocking wizard a zároveň vstupem entity nejvyšší úrovně je hodinový signál o požadované frekvenci. Entita nejvyšší úrovně má celkem tři výstupy, které jsou napojeny na výstup celého systému, jedná se o vektory `col[7:0]`, `led[3:0]` a `row[7:0]`.



Obrázek 1: Blokový návrh systému

4 Použité konstanty

Celý systém aplikace používá několik konstant a uživatelem definované typy, které usnadňují pozdější orientaci ve zdrojovém kódu.

4.1 Uživatelský typ

Systém je řízen vnitřním konečným stavovým automatem. Pro tyto účely je definován vlastní uživatelský typ `FSM_STATE`, který obsahuje všechny potřebné stavy FSM (`STATE_NORMAL`, `STATE_INVERSE`, `STATE_IMAGE` a `STATE_ANIMATION`).

4.2 Konstanty buněk

Nedílnou součástí jsou definice konstant vektorů aktuálního stavu a nastavení jednotlivých buněk systému pro každou kombinaci dekadických čítačů (`CELL_{00-30}`). Hodnota těchto vektorů je následně přiřazena do ROM paměti podle odpovídajícího stavu systému a konečného stavového automatu.

4.3 Konstanta obrázku

Další konstantou je 64bitový vektor `CELL_IMAGE` obsahující požadovaný stav buněk pro případné zobrazení vlastního obrázku. Tento vektor je neměnný, takže je vhodné ho zvolit jako konstantu.

4.4 Konstanty čítačů

Systém jako celek využívá velké množství čítačů, které řídí chod aplikace. Nyní je nutné odůvodnit nastavení jednotlivých konstant těchto čítačů pro snadné pochopení činnosti.

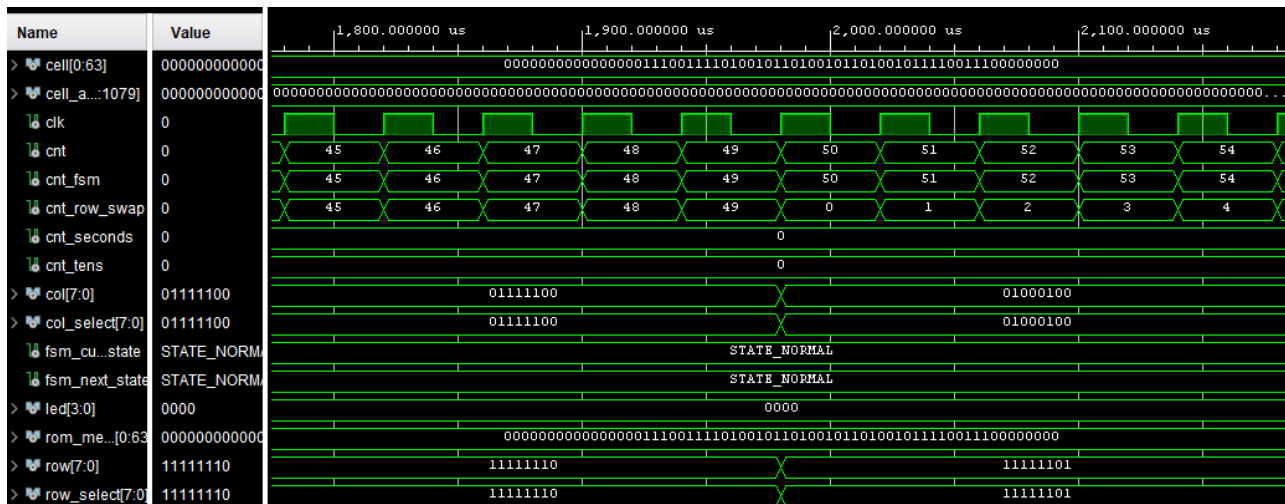
- `CNT_ANIMATION_MAX` – konstanta ovlivňující maximální hodnotu čítače, který slouží k ovládání animace, přesněji efektu tzv. běžícího textu. Její hodnota $2.600.000 - 1$ udává počet náběžných hran hodinového signálu mezi dvěma různými pozicemi zobrazovaného textu, což při frekvenci 25 MHz odpovídá **104 ms**.
- `CNT_FSM_ANIMATION` – konstanta udávající počet taktů hodinového signálu, kdy se konečný stavový automat přepne do stavu `STATE_ANIMATION`. $900.000.000 - 1$ je počáteční hodnota této konstanty a při dané frekvenci je to v reálném čase **36 s**. Od tohoto okamžiku tudíž probíhá na displeji animace.
- `CNT_FSM_IMAGE` – konstanta pro přechod konečného stavového automatu do stavu `STATE_IMAGE` s původní hodnotou $775.000.000 - 1$. Díky tomu je FSM schopen detekovat požadovaný přechod v čase **31 s**.
- `CNT_FSM_MAX` – konstanta, která má za úkol omezit celkový počet náběžných hran hodin pro jedno opakování smyčky na $1.250.000.000 - 1$. Jednoduše se jedná o moment, kdy se celé zobrazování ukončí a pokračuje se znovu od začátku (konkrétně v čase **50 s**).
- `CNT_FSM_INVERSE` – konstanta, díky které je schopen konečný stavový automat přejít do stavu `STATE_INVERSE`. Hodnota $525.000.000 - 1$ po přepočtu značí ve skutečnosti čas **21 s**.
- `CNT_MAX` – konstanta, jejíž hodnota $25.000.000 - 1$ udává počet taktů hodinového signálu za jednu sekundu (**1 s**). S její pomocí je poté daleko snazší počítání jednotlivých sekund čítačů pro desítky a sekundy.
- `CNT_ROW_SWAP_MAX` – konstanta odpovídající rovněž uvedenému počtu náběžných hran hodinového signálu ($50.000 - 1$). Jakmile čítač dosáhne této hodnoty, změní se aktuálně zobrazovaný řádek na následující. Tímto způsobem je zobrazování implementováno pomocí tzv. *multiplexingu* a změna nastává každé **2 ms**.

5 Simulace

Součástí realizovaného řešení je nezbytné využití simulace, která značně urychluje proces dokončení aplikace v rozumném čase a zvyšuje se efektivita výsledného programu.

5.1 Zobrazování

Detailní obrázek ze simulace naznačuje funkčnost zobrazování. Lze si povšimnout měnících se hodnot signálů `col[7:0]` a `col_select[7:0]` pro výběr zobrazovaného sloupce a současně signálů `row[7:0]` a `row_select[7:0]` sloužící k výběru požadovaného řádku.



Obrázek 2: Simulace zobrazování

5.2 FSM řadič

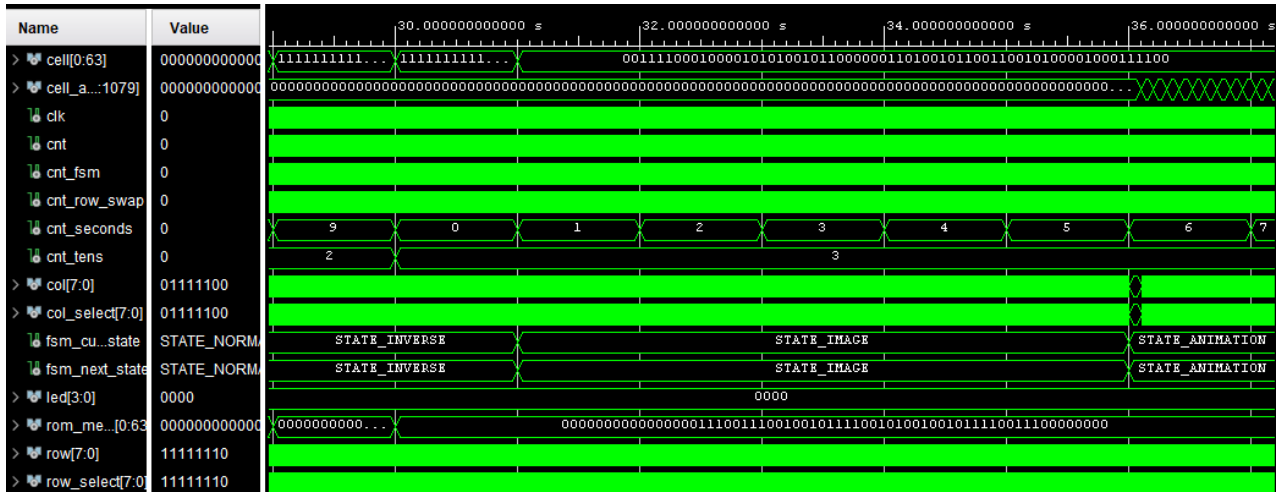
Druhý screenshot ze simulace zobrazuje všechny definované stavy konečného stavového automatu a přechody mezi nimi v očekávaných časech, jelikož simulován byl skutečný čas aplikace, která i v reálném čase běží 50 s.



Obrázek 3: Simulace FSM řadiče

5.3 ROM paměť

Dalším příkladem ze simulace je demonstrace přechodu systému z inverzního zobrazování dvouciferných dekadických čísel do stavu zobrazení obrázku a následně do posledního stavu vlastní animace. Signál `cell[0:63]` nejprve ve stavu `STATE_INVERSE` obsahuje invertovaný obsah vektorového signálu `rom_memory[0:63]`, ve stavu `STATE_IMAGE` má konstantní hodnotu pro zobrazení obrázku a konečně ve stavu `STATE_ANIMATION` je využíván signál `cell_animation[0:1079]`, který se mění každých 104 ms.



Obrázek 4: Simulace ROM paměti

5.4 Celý systém

Při pohledu na snímek simulace celého systému je zřejmé, že každý signál se chová odpovídajícím způsobem a vše dohromady tvoří funkční aplikaci.



Obrázek 5: Simulace celého systému

6 Video

Video funkční implementace s komentářem je možné najít zde: <https://youtu.be/30gyfTxsS0>.